Smart Parking System

Interactive 2D/3D Parking Visualization,
Booking & Navigation System

Computer Graphics Laboratory Project

Language: Pure C (C99)

Graphics: OpenGL | GLUT (FreeGLUT) | GLU

Platform: Windows | MinGW-w64 GCC

Project Overview

A real-time 3D parking lot simulator built in C using OpenGL, where users can visually select parking slots, book them with customer details, watch a car navigate to the spot via A pathfinding, and manage timed parking -- all inside a single graphical window.*

Imagine you walk into a smart parking lot at a mall. There is a big screen showing you which spots are free, which are taken, which are reserved, which are for disabled persons, and which have EV chargers. This project IS that screen -- but interactive and in 3D.

With this system, you can:

- **Click on a parking spot** to select it
- **Press 'B'** to book it (enter name, phone, vehicle number)
- **Watch a car drive** from the entrance gate to your spot (animated!)
- **See a countdown timer** ticking down your parking time
- **Get warnings** when time is almost up (< 10 min, < 5 min)
- **Release the spot** when done by pressing 'R'

Slot Color Coding

Color	Status	Meaning
Green	AVAILABLE	Slot is free and ready for booking
Red	OCCUPIED	Vehicle parked, timer running
Yellow	RESERVED	Pre-reserved, cannot be booked
Blue	ACCESSIBLE	For disabled persons (eligibility check)
Purple	EV CHARGING	Electric vehicle charging station
Magenta (Flashing)	TIME EXPIRED	Timer hit zero, awaiting release

Project Architecture

The project is written in pure C language using OpenGL for graphics. It is organized into 6 source modules, each handling a specific responsibility:

Module	File	Responsibility
Entry Point	main.c	Keyboard/mouse input, view switching, orchestration
Parking Logic	parking.c	36 slots, status management, timers, customer data
Graphics Engine	graphics.c	All 2D/3D rendering, HUD, booking wizard UI
Navigation	navigation.c	A* pathfinding from entrance to selected slot
Vehicle	vehicle.c	3D car model drawing and movement animation
Camera	camera.c	3D orbital camera controls (zoom, rotate, orbit)

Computer Graphics Concepts Demonstrated

1. 2D and 3D Rendering (Dual View Modes)

2D View uses glOrtho() for orthographic projection -- a flat top-down map view, similar to Google Maps. 3D View uses gluPerspective() for perspective projection -- objects farther away appear smaller, like a video game.

2. Geometric Transformations

glTranslatef() moves objects to their position in the world. glRotatef() rotates the car as it turns corners along the navigation path. glScalef() sizes objects up or down. glPushMatrix() / glPopMatrix() save and restore transformation states, enabling hierarchical modeling (e.g., car body + wheels as a single unit).

3. Drawing Primitives (2D & 3D)

GL_QUADS for parking slots and buildings. GL_TRIANGLES for rooftops and directional arrows. GL_LINES for grid lines and road markings. GL_LINE_STRIP for the cyan navigation path. gluCylinder / gluSphere for 3D car wheels and spherical shapes.

4. Camera System (gluLookAt)

gluLookAt(eye, target, up) positions the virtual camera in 3D space. Orbital math using trigonometric functions (sin, cos) allows the user to rotate the camera around the parking lot using arrow keys, and zoom in/out with W/S keys.

5. OpenGL Lighting

glEnable(GL_LIGHTING) activates OpenGL's built-in lighting engine. glLightfv() configures light position, ambient/diffuse/specular colors. GL_COLOR_MATERIAL makes objects respond to light based on their assigned color. glShadeModel(GL_SMOOTH) enables smooth (Gouraud) shading across polygons.

6. Mouse Picking (Click Detection in 3D)

gluUnProject() converts a 2D screen-space mouse click coordinate into 3D world coordinates. This uses ray-plane intersection to detect which parking slot the user clicked on, working correctly in both 2D and 3D views.

7. Non-blocking Timer Engine

glutTimerFunc(16, ...) creates a 60 FPS rendering loop for smooth vehicle animation. glutTimerFunc(1000, ...) creates a per-second callback for parking countdown timers. Both run without blocking the main rendering loop, ensuring the UI stays responsive.

Key Features

Feature	Description
36 Parking Slots	Grid layout with Standard, Disabled, EV, and Reserved types
2D / 3D Toggle	Press '2' for top-down map, '3' for 3D perspective view
Booking Wizard	Press 'B' to enter name, phone, vehicle plate, type, duration
A* Navigation	Car finds shortest path from entrance to slot and drives there
Countdown Timers	Real-time countdown with warnings at <10 min and <5 min
Time Expiry	Slot flashes magenta when timer reaches zero
Slot Release	Press 'R' to free up a spot and reset all data
Live Dashboard	Real-time stats: available, occupied, reserved, expired counts
Customer Info	Press 'I' to view customer details and remaining time
Lighting Toggle	Press 'L' to toggle OpenGL lighting on/off

Keyboard Controls

Key	Action
2	Switch to 2D Top-Down View (Orthographic)
3	Switch to 3D Perspective View
B	Book Selected Available Slot (Opens Booking Wizard)
N	Start A* Navigation to Selected Slot
I	View Customer / Booking Information Overlay
T	Select Test Timer (30s / 60s for Lab Demo)
P	Pause / Resume Parking Timer or Vehicle Navigation
R	Release Slot (Remove car, clear details, reset timer)
A / D	Add / Remove Vehicle
L	Toggle OpenGL Lighting On / Off
W / S	Zoom Camera In / Out
Q / E	Adjust Camera Height Up / Down
Arrow Keys	Orbit Camera View (Yaw / Pitch)
ESC	Exit Application / Cancel Modal

Lab Demonstration Steps

- **Step 1:** Run smart_parking.exe to launch the application.
- **Step 2:** Click on a green (available) slot, e.g., A05, to select it.
- **Step 3:** Press 'B' to open the booking wizard and enter customer details.
- **Step 4:** Confirm booking -- the vehicle starts navigating along the cyan A* path.
- **Step 5:** Watch the car animate from the entrance gate to the selected slot.
- **Step 6:** Once parked, the slot turns red and the countdown timer begins.
- **Step 7:** Press 'I' to view customer info, vehicle plate, and remaining time.
- **Step 8:** Wait for the timer to expire -- the slot flashes magenta.
- **Step 9:** Press 'R' to release the slot -- it turns green again, stats update.
- **Step 10:** Press '3' to switch to 3D view. Use arrow keys to orbit the camera.
- **Step 11:** Press 'L' to toggle lighting on/off and demonstrate the lighting model.

Technology Stack

Component	Technology
Programming Language	C (C99 Standard)
Graphics API	OpenGL + GLUT (FreeGLUT) + GLU
Compiler	MinGW-w64 GCC
Platform	Microsoft Windows
Pathfinding Algorithm	A* (A-Star) Search
Build System	build.bat (single-click compile)

Real-World Application

Smart parking systems are widely used in malls, airports, hospitals, and smart cities. This project simulates the same concept using Computer Graphics, demonstrating how CG principles -- transformations, projections, lighting, camera systems, and pathfinding -- can be applied to practical, real-world problems.